

CPSC 375

Introduction to Data Science and Big Data Analytics

Dr. Kanika Sood
kasood@fullerton.edu

What we will cover today

- ClassWork
- Data cleaning/wrangling
 - In Python with Pandas

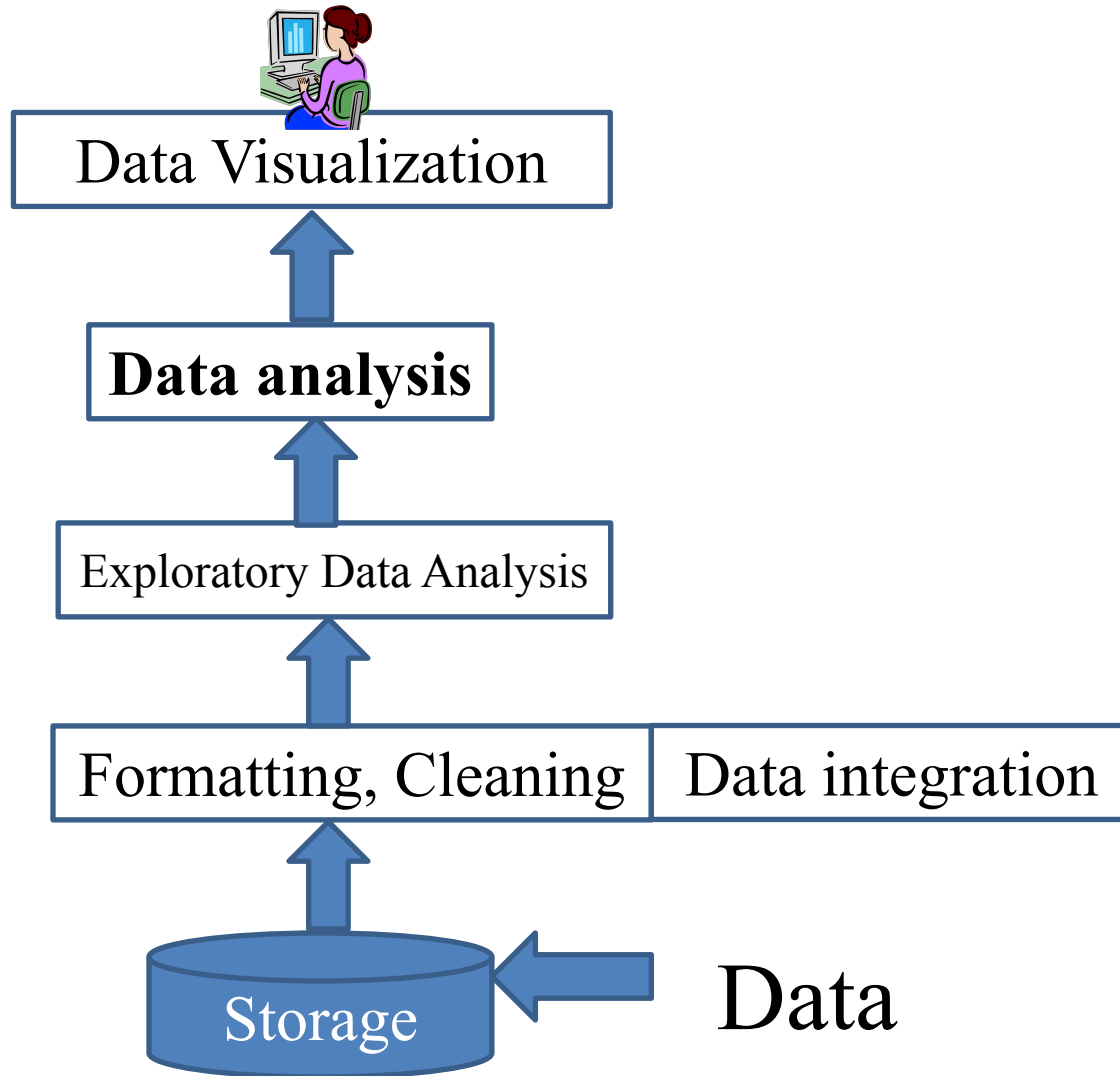
1. Exploratory data analysis	B1: Ch. 3-4 B1: Ch. 7 B1: Ch. 9 B1: Ch. 13
2. Data wrangling	B1: Ch. 4
3. Tidy data	B1: Ch. 22

Welcome	
Introduction	>
Quick Start	▼
Whole Game	
Data Visualisation	
Workflow: Basics	
Data Transformation	
Workflow: Style	
Tidy Data	
Workflow: Writing Code	
Data Import	
Workflow: Packages and Environments	
Postscript: Getting Further Help	
Visualise	▼
Visualisation	
Layers	
Exploratory Data Analysis	
Graphics for Communication	
Transform	▼
Boolean Data	
Numbers	
Strings and Text	
Regular Expressions, aka regex	
Categorical Data	
Dates and Times	
Missing Values	
Joins	

An exercise in Python

- From the iris dataset, calculate the mean Sepal.Length and mean Sepal.Width for those flowers with Petal.Length > 3.0
- Hard!

The Data Science Process



- “80 percent of data science is cleaning the data and ...

- “80 percent of data science is cleaning the data and 20 percent is complaining about cleaning the data”

- Kaggle founder and CEO Anthony Goldbloom:

“... data cleaning is a *much* higher proportion of data science than an outsider would expect. Actually training models is typically a relatively small proportion (less than 10 percent) of what a machine learner or data scientist does.”

- Survey to data scientists about the biggest barriers faced at work
 - the most common answer: “dirty data”

Data wrangling

- **Data wrangling:**
 - *transforming* data from one form into another form with the intent of making it more suitable for later analysis
- Also called **data munging**

Data wrangling in Python and R

- NumPy logical indexing can be used to wrangle data
 - Bracket-based subsetting
- R includes `tidyr` and `dplyr` packages developed by Hadley Wickham
 - Part of the the `tidyverse` package
 - Called “verbs” – the language of data cleaning
 - A grammar of data manipulation
 - Python ports: `dfply`, `dplython`, `plydata`, `pandas-ply`, `datar`, `siuba`, ...

Data wrangling in Pandas

- NumPy logical indexing can be used to wrangle data
 - Bracket-based subsetting
 - Overloads bitwise operators (&, |, ~) to do logical operations
- A more complete set of functions in Pandas
 - Allows standard Python syntax
 - Logical operators
 - Method chaining
 - Some functionality similar to Tidyverse (e.g. `wide_to_long()` vs. `pivot_longer()`)

Core data wrangling methods

- `.query()`: remove rows (observations)
- `.filter()`, `.drop(,axis=1)`: remove columns (variables)
- `.sort_values()`: sort the observations
- `.assign()`, `.transform()`: add or modify variables
- `.agg()`: collapse multiple values into a single value
- `.groupby()`: divide dataset into subgroups
- [Data Wrangling with pandas Cheat Sheet](#)

.query()

- subset observations based on their values
- first argument:
 - string containing an expression that filters the data frame
 - "slightly modified Python syntax by default"
 - treats bitwise operators (&, |, ~) as logical
 - Same precedence as logical and, or, not
- optional argument:
 - `parser='python'` enforces strict Python semantics

mpg dataset

```
>>> mpg = sm.datasets.get_rdataset('mpg', 'ggplot2').data
>>> mpg.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 234 entries, 0 to 233
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   manufacturer    234 non-null   object
 1   model           234 non-null   object
 2   displ           234 non-null   float64
 3   year            234 non-null   int64
 4   cyl             234 non-null   int64
 5   trans           234 non-null   object
 6   drv             234 non-null   object
 7   cty             234 non-null   int64
 8   hwy            234 non-null   int64
 9   fl              234 non-null   object
10  class           234 non-null   object
...
```

.query()

- Multiple conditions with & or and
 - `mpg.query("cty > 25 & hwy > 27")`
 - `mpg.query("cty > 25 and hwy > 27")`
- | or or
- Some convenience operators:
 - `mpg.query("manufacturer=='honda' | manufacturer=='toyota'")`
 - `mpg.query('manufacturer in ["honda", "toyota"]')`

.filter() and .drop()

- Get column subset using operations based on the names of the variables
- `df.filter(...)`, `df.filter(items=...)`:
 - a list of column names to select
- `df.drop(..., axis=1)`
 - a list of column names to drop

String helper functions and filter arguments

- To specify column names without listing every name explicitly
 - `mydata.loc[:, mydata.columns.str.startswith("abc")]`
 - matches names that begin with “abc”.
 - `mydata.filter(regex="xyz$")`
 - matches names that end with “xyz”.
 - `mydata.filter(like="ijk")`
 - matches names that contain “ijk”.

Method chaining

- Most Pandas DataFrame methods return a new DataFrame object
- This means you can call DataFrame methods on the result of a DataFrame method
- This syntax makes multi-step chains easier to see

```
>>> mpg.query('cty > 25').filter(["manufacturer"])
```

Classwork

- [soodk6/cpsc375-classwork](#)
 - wrangling-pandas.ipynb – Part 1

Use the query(), filter()

.sort_values()

- changes the order of rows (sorting)
- Parameters:
 - column name to sort by, or lambda expression to apply to column name with `key=`, or
 - list of column names; additional columns used to break ties in the values of preceding columns
- Ascending order by default:
 - `mpg.sort_values('cty')`
- Descending order:
 - `mpg.sort_values(['cty'], ascending=False)`

Lambda expressions

- Anonymous, one-line functions
 - `lambda args: expression`
 - **equivalent to `def fn(args): return expression`**
 - but doesn't define a new function `fn`
 - Key features:
 - Single expression, **no function name**
 - Often used as arguments to other functions
- ```
>>> mpg.sort_values('cty', key=lambda cty: -cty)
```

# Classwork (.sort\_values)

- [soodk6/cpsc375-classwork](#)  
–wrangling-pandas.ipynb – Part 2

# .assign()

- add new columns that are functions of existing columns

```
>>> mpg.assign(mileage=(mpg.cty+mpg.hwy)/2)
>>> mpg.assign(mileage=lambda df:(df.cty+df.hwy)/2)
 – always adds new columns at the end of the dataset
 – returns new dataset
```

- can also use **assignment statement** directly

```
>>> mpg['mileage'] = (mpg.cty+mpg.hwy) / 2
 – changes existing dataset in-place
 – requires df['new_col'], not df.new_col syntax
```

# .agg()

- Collapses a data frame to a **single** row
- E.g.:
  - By summing or taking the mean of a column
- `airquality['Wind'].agg('mean')`
- `airquality.agg({'Wind': 'count'})`
  - Available functions listed as “computations / descriptive stats” under [GroupBy](#)

# .groupby()

- Divides data into non-overlapping groups
- Divide dataset according to one or more categorical variables

```
cars_by_manuf = mpg.groupby('manufacturer')
```

- Original dataset had **one group** (the whole data)
- Transformed object has **one group for every unique** manufacturer

# .groupby + .agg

- Together, collapses a data frame to one row **per group**
- similar to R base's aggregate()

# `.group_by + .agg`

- How has the cty mileage changed over the years?
  - Calculate the mean cty for every year
- How has the cty mileage changed over the years for every manufacturer?
  - Calculate the mean cty for every year and for every manufacturer

# nyflights dataset

```
>>> flights = sm.datasets.get_rdataset('flights', 'nycflights13').data
>>> flights.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 336776 entries, 0 to 336775
```

```
Data columns (total 19 columns):
```

| #  | Column         | Non-Null Count  | Dtype   |
|----|----------------|-----------------|---------|
| 0  | year           | 336776 non-null | int64   |
| 1  | month          | 336776 non-null | int64   |
| 2  | day            | 336776 non-null | int64   |
| 3  | dep_time       | 328521 non-null | float64 |
| 4  | sched_dep_time | 336776 non-null | int64   |
| 5  | dep_delay      | 328521 non-null | float64 |
| 6  | arr_time       | 328063 non-null | float64 |
| 7  | sched_arr_time | 336776 non-null | int64   |
| 8  | arr_delay      | 327346 non-null | float64 |
| 9  | carrier        | 336776 non-null | object  |
| 10 | flight         | 336776 non-null | int64   |

```
...
```

# Classwork

- [soodk6/cpssc375-classwork](#)  
–wrangling-pandas.ipynb – Part 3

# Other data wrangling methods

- .take(): Subset rows based on row number
- .nlargest(): Get top  $n$  rows by a variable value
- .join: Combine datasets on matching variables
- .rename(): Change the names of variables
- reset\_index(): Remove grouping from data
- drop\_duplicates(): Keep observations that are non-redundant (cf. *unique*)

# .take

- Subset rows based on row number

```
airquality.take(range(0, 5))
```

```
airquality.take(range(1, 5, 2))
```

# nlargest(), nsmallest()

- Get top  $n$  rows sorted by variable  $wt$
- `....nlargest( $n$ ,  $wt$ )`

```
airquality.nlargest(1, 'Solar.R')
```

# Classwork

- [soodk6/cpsc375-classwork](#)  
– wrangling-pandas.ipynb – Part 4

# Mentimeter



# Announcements

- Class Project: Released
- Groups of 3

| Project |   |                                             | ✓ | + | ⋮ |
|---------|---|---------------------------------------------|---|---|---|
| ⋮       | 📎 | Health Sciences Data File New (project).csv | ✓ |   | ⋮ |
| ⋮       | 📎 | Data dictionary project.csv                 | ✓ |   | ⋮ |
| ⋮       | 📄 | Draft submission                            | ✓ |   | ⋮ |
| ⋮       | 📄 | Final project submission                    | ✓ |   | ⋮ |
| ⋮       | 📎 | CPSC375SummerClassProjectRequirements.pdf   | 🔒 | ✓ | ⋮ |